Simulador de Gestor de Base de Datos

BASE DE DATOS 2
PROYECTO 1

Integrantes:

- Jorge Sebastian Tenorio Romero
- Sebastian Romero Pahuara
- Carlos Alberto Villegas Arce
- Llorent Eloy Nunayalle Brañes
- Paris Lenard Herrera Torres

Sábado 9 de mayo de 2026

CATALOG.PY

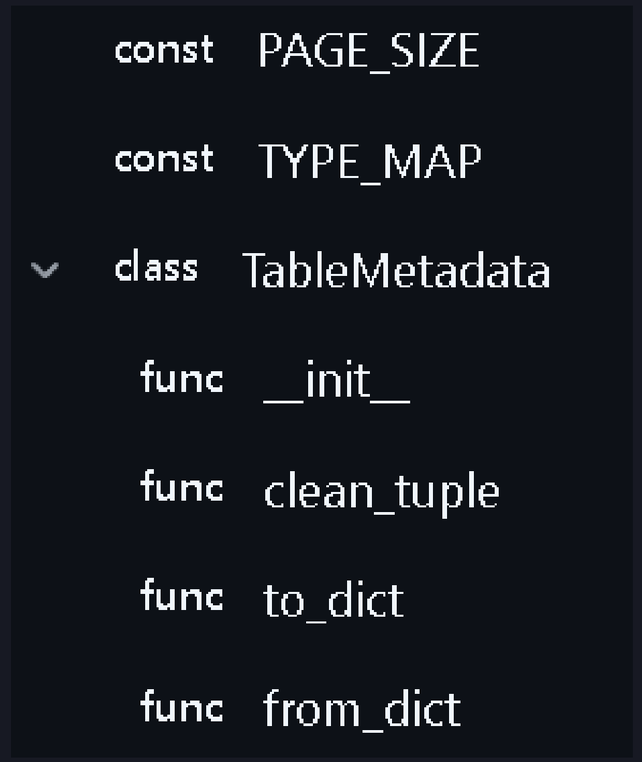
Clase: TableMetadata

Atributos:

- name
- columns
- key_column
- index_tech
- struct_format
- record_size
- block_factor
- PAGE_SIZE = 4096

Tipos soportados:

- int
- float
- Varchar (50 bytes)
- Boolean
- Date
- Point



Persiste como system_catalog.json

Executor lo carga al iniciar

EXECUTOR.PY

Clase: Executor

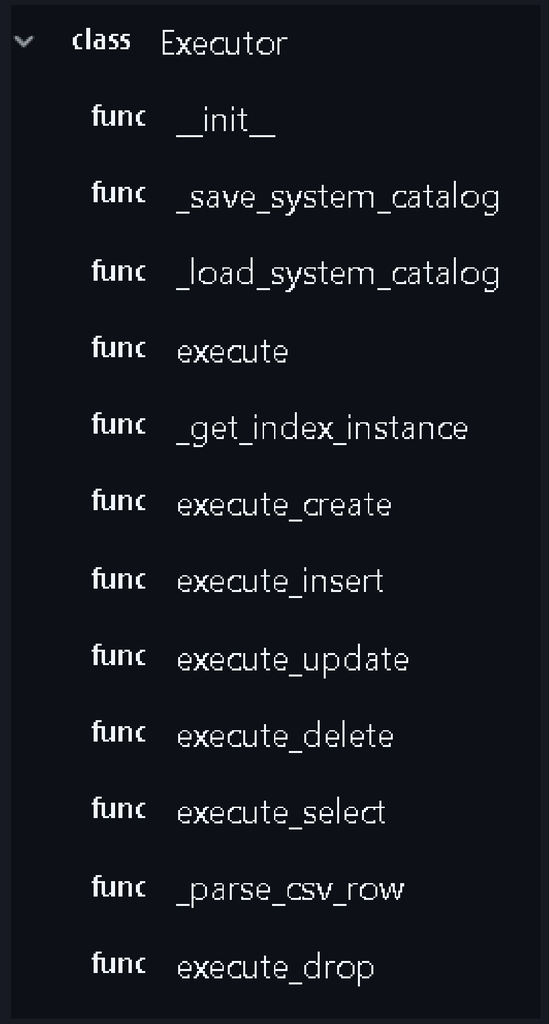
Atributos:

- self.catalog (diccionario)

Métodos:

- execute(ast)
  - execute_create()
  - execute_select()
  - execute_insert()
  - execute_delete()
  - execute_update()
  - execute_drop()
- _get_index_instance()

_get_index_instance() instancia la clase correcta según el INDEX declarado



BASEINDEX.PY

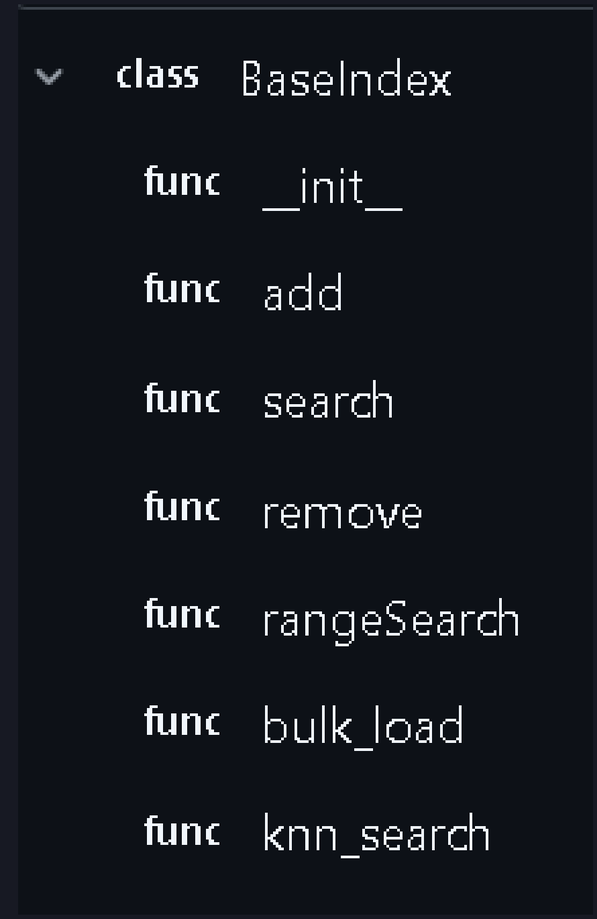
Clase: BaseIndex

Atributos:

- disk_reads
- disk_writes

Interfaz:

- add
- search
- remove
- rangeSearch
- bulk_load
- knn_search



SEQUENTIAL.PY

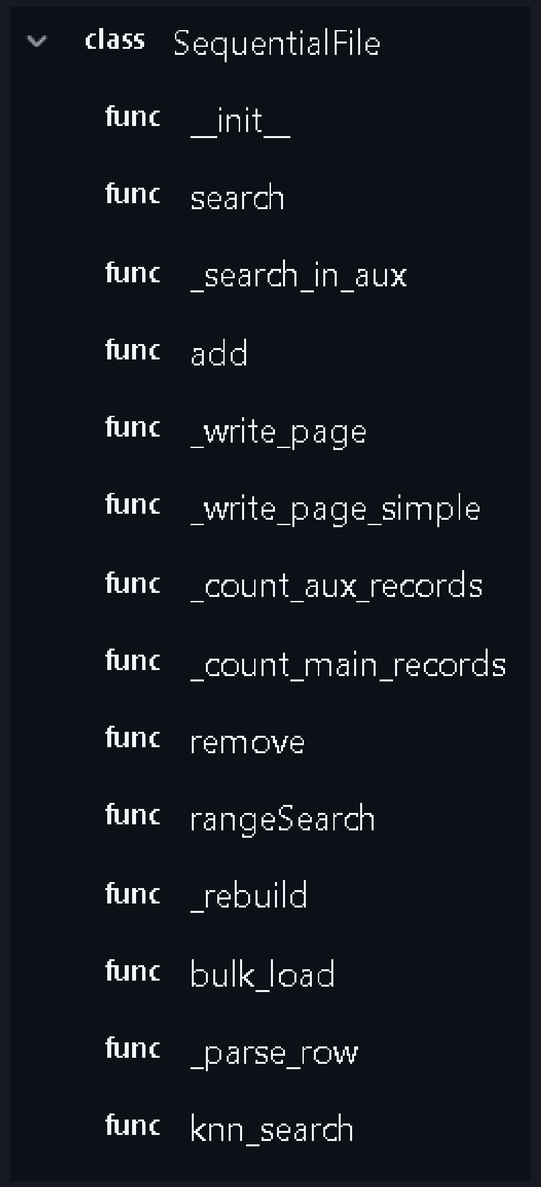
Clase: SequentialFile

Archivos en disco:

- _main.dat
- _aux.dat

Operaciones:

- add
  - → _rebuild()
- search
  - → búsqueda binaria
- remove
  - → marcado
- rangeSearch
  - → búsqueda binaria
- bulk_load
  - → External Sort



HASH.PY

Clase: Extendible Hashing

Archivos en disco:

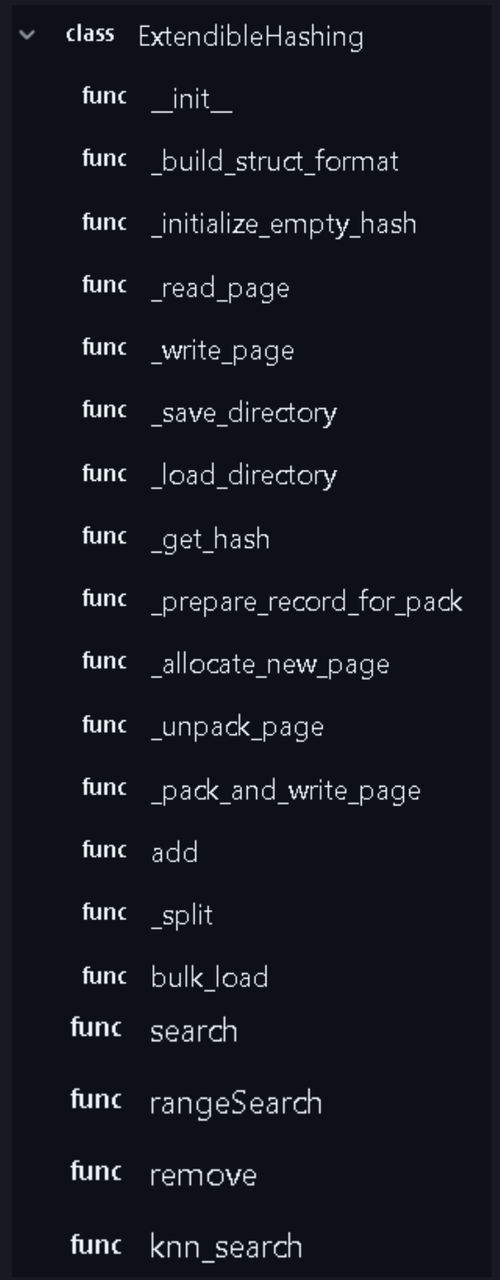
- _hash_dir.dat
- _hash_buckets.dat

Atributos:

- global_depth
- directory[]
- MAX_RECORDS
- key_index

Operaciones:

- add → split
- search
- remove → marcado
- bulk_load



BPLUSTREE.PY

Clase: BPlusTree, BPlusNode

Archivos en disco

- _bplustree.dat
- .heap

Atributos - BPlusTree

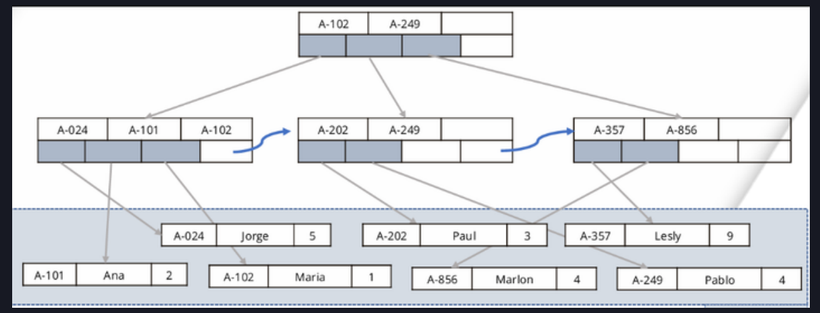
- ORDER
- Key_fmt
- Key_size
- Key_type
- filename
- heap

Atributos BPlusNode

- node_type
- is_root
- num_keys
- parent_ptr
- next_leaf
- Keys[], pointers[]
- Header (BBiii)

Operaciones:

- add
  - → split de hoja
  - → propagación al padre
- search
  - → descenso y lookup en heap
- remove
  - → borra hoja + heap
- range_search
  - → begin_key
- bulk_load
  - genera entradas → ordena → construye el árbol



class BPlusNode
const HEADER_FORMAT
const HEADER_SIZE
const INT_SIZE
func __init__
func pack_header
func unpack_header
func pack
func unpack

class BPlusTree
const HEADER_FORMAT
const HEADER_SIZE
func __init__
func _read_header
func _write_header
func _read_node
func _write_node
func _is_numeric
func _normalize_key_value
func _compare_keys
func _binary_search_internal
func _binary_search_leaf
func _min_keys_leaf
func _min_keys_internal
func _find_first_leaf
func search_one
func search_all
func search
func range_search_1
func rangeSearch
func insert
func add
func _insert_in_leaf
func _insert_in_internal
func _insert_recursive
func _split_leaf
func _split_internal
func delete
func remove
func _delete_recursive
func _remove_from_leaf
func _remove_from_internal
func _binary_search_internal_left...
func _update_parent_separator_...
func _rebalance_leaf

HEAPFILE

Clase: HeapFile

Atributos

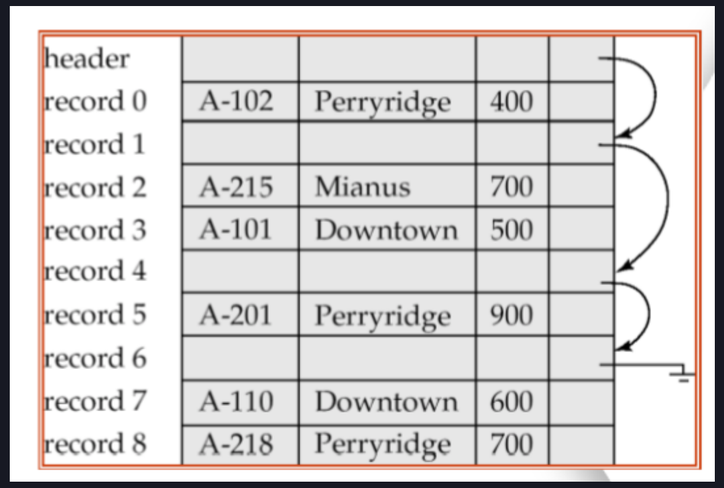
- File_Header_Format
- File_Header_Size
- Record_Meta_Format
- Page_Header_Format

Metodos

- _read_header() / _write_header()
- _page_offset(page_id)
- _record_offset_in_page(slot)

Operaciones

- insert(record_tuple)
- delete(offset)
- read(offset)
- read_many(offsets)
- bulk_insert(records)
- _write_page()
- _write_page_raw()




```
class HeapFile
  const FILE_HEADER_FORMAT
  const FILE_HEADER_SIZE
  const RECORD_META_FORMAT
  const RECORD_META_SIZE
  const PAGE_HEADER_FORMAT
  const PAGE_HEADER_SIZE
  func __init__
  func _read_header
  func _write_header
  func _page_offset
  func _record_offset_in_page
  func insert
  func delete
  func read
  func read_many
  func bulk_insert
  func _write_page
  func _write_page_raw
```

RTREE.PY

Clase: RtreeIndex

Archivos en disco

- _rtree.dat
- _main.dat + _aux.dat

Atributos

- TreeFile
- data_storage
- M
- m
- entry_fmt
- header_fmt

Clases Auxiliares

- Point(x ,y, pk)
  - → distance_to(other)
- MBB(min_x, min_y, max_x, max_y)
  - → area(), distance_to(point)

Operaciones:

- rangeSearch(point, radius)
  - DFS
- knn_search(point, k)
  - → Best-First con min hea
- Bulk-load
  - → Algoritmo STR

PARSER SQL

sql_parser.py
clase: DBMSSqlParser

DBMSSqlParser:

- Descenso Recursivo: Cada produccion de la gramática es un método de python

Sentencias:

- CREATE
- SELECT
- BETWEEN
- IN POINT RADIUS
- IN POINT K
- INSERTE
- DELETE
- UPDATE
- DROP

sql_lexer.py
clase: DBMSSqlLexer

DBMSSqlLexer:

- Analizador léxico de consulta, en tokens

```
class Token
    const type
    const value
    const line
    const column
class DBMSSqlLexer
    func __init__
    func tokenize
```

```
class DBMSSqlParser
    func __init__
    func current_token
    func match
    func parse
    func parse_statement
    func parse_create
    func parse_column_list
    func parse_select
    func parse_condition
    func parse_insert
    func parse_delete
    func parse_drop
    func parse_update
```

PARSER SQL GRAMÁTICA

```
<S>             ::= <CREATE_STMT> | <SELECT_STMT> | <INSERT_STMT>
                  | <DELETE_STMT> | <DROP_STMT>

<CREATE_STMT>   ::= CREATE TABLE <ID> ( <COL_DEF_LIST> ) [ FROM FILE <STRING> [ DELIMITER <STRING> ] ] ;
<COL_DEF_LIST>  ::= <COL_DEF> | <COL_DEF> , <COL_DEF_LIST>
<COL_DEF>       ::= <ID> <TYPE> [ INDEX <ID> ]
<TYPE>          ::= INT | FLOAT | VARCHAR | POINT | BOOLEAN | DATE

<SELECT_STMT>   ::= SELECT * FROM <ID> <SELECT_OPTS> ;
<SELECT_OPTS>   ::= GROUP BY <ID>
                  | WHERE <ID> <CONDITION> [ ORDER BY <ID> ]
<CONDITION>     ::= = <VALUE>
                  | BETWEEN <NUMBER> AND <NUMBER>
                  | IN ( POINT ( <NUMBER> , <NUMBER> ) , <SPATIAL_PARAM> )
<SPATIAL_PARAM> ::= RADIUS <NUMBER> | K <NUMBER>

<INSERT_STMT>   ::= INSERT INTO <ID> VALUES ( <VALUE_LIST> ) ;
<VALUE_LIST>    ::= <VALUE> | <VALUE> , <VALUE_LIST>

<DELETE_STMT>   ::= DELETE FROM <ID> WHERE <ID> = <VALUE> ;

<DROP_STMT>     ::= DROP TABLE <ID> ;

<VALUE>         ::= <NUMBER> | <STRING>
```

CONTROL DE CONCURRENCIA Y TRANSACCIONES

transaction_manager.py
Clase: TransactionManager

Aislamiento: Bloqueo de registros específicos

Atributos:

- Executor
- record-locks (diccionario)
- _manager_locks
- lock_owners
- waiting_txs

Métodos:

- _get_lock: Prevención de deadlock
- execute_transaction
  - → procesa una transacción completa

main.py
Api: /simulate-concurrency

La concurrencia se logra ejecutando transacciones simultáneas mediante hilos

```
const app
class QueryRequest
  const query
class TransactionPayload
  const tx_id
  const queries
class ConcurrencyRequest
  const transactions
func parse_query
func get_schema
func execute_query
func simulate_concurrency
```

EVALUACIÓN EXPERIMENTAL

Tabla 1: Operación de Inserción

Técnica	Tamaño (N)	Accesos a Disco (Reads + Writes)	Tiempo de Ejecución (ms)
Sequential File	1,000	2	196.05
	10,000	2	227.91
	100,000	2	749.85
B+ Tree	1,000	3	193.37
	10,000	3	256.54
	100,000	3	781.80
Extendible Hashing	1,000	5	197.05
	10,000	2	302.18
	100,000	2	817.00

Tabla 2: Búsqueda Puntual (Point Query)

Técnica	Tamaño (N)	Accesos a Disco (Páginas)	Tiempo de Ejecución (ms)
Sequential File	1,000	1	290.61
	10,000	10	234.42
	100,000	13	736.31
B+ Tree	1,000	2	195.68
	10,000	2	247.56
	100,000	2	824.33
Extendible Hashing	1,000	1	179.18
	10,000	1	273.95
	100,000	1	766.55

Tabla 3: Búsqueda por Rango (Range Query)

Técnica	Tamaño (N)	Accesos a Disco (Páginas)	Tiempo de Ejecución (ms)
Sequential File	1,000	14	237.06
	10,000	19	242.98
	100,000	21	840.50
B+ Tree	1,000	2	203.48
	10,000	2	246.89
	100,000	2	752.56

Nota: El Extendible Hashing no se incluye en esta tabla ya que su arquitectura no soporta búsquedas por rango de forma nativa.

Tabla 4: Consultas Espaciales (R-Tree)

Operación	Tamaño (N)	Accesos a Disco (Páginas)	Tiempo de Ejecución (ms)
Búsqueda por Radio	1,000	1,704	915.91
	10,000	42,481	12,009.56
	100,000	468,296	120,193.01
K-Nearest Neighbors (KNN)	1,000	84	274.20
	10,000	129	425.97
	100,000	179	1,479.87